



Name:- JayKishan J Saharan

Div:- D Roll no.:- 26.

Course Program:- BTech CSE SY.

Subject:- DSA.

### DSA Lab assignment 3

08/19

\* Title:- Write a c++ Program to sort an array of N elements using Insertion Sort and Merge Sort.

Q.1) What is Insertion sort and how does it work?

Sol<sup>n</sup>:- Insertion Sort is a simple and intuitive sorting algorithm that builds final sorted array one item at a time.

Working / Algorithm:-

Step-1) Starts from second element (index=1) of array.

Step-2) compare current element with elements in sorted portion (to the left).

(-3) Shift larger element to the right to make space for the current element.

Step-4) Insert the current element into its correct position.

Step-5) Repeat until entire array is sorted.

Q.2) Sort the elements [64, 25, 12, 22, 11] using Insertion Sort.

Sol<sup>n</sup>:-1) Array = [64, 25, 12, 22, 11]

i=1, Key=25

compare 25 with 64, shift 64 : [64, 64, 12, 22, 11]

Insert 25: [25, 64, 12, 22, 11]



2) Next iteration: [25, 64, 12, 22, 11]

$i=2$ , Key = 12

compare 12 with 64 and 25, Shift 64, 25: [25, 25, 64, 22, 11]

Insert 12: [12, 25, 64, 22, 11]

3) Next iteration: [12, 25, 64, 22, 11]

$i=3$ , Key = 22

compare 22 with 64, Shift 64: [12, 25, 64, 64, 11]

Compare 22 with 25, Shift 25: [12, 25, 25, 64, 11]

Insert 22: [12, 22, 25, 64, 11]

4) Next iteration: [12, 22, 25, 64, 11]

$i=4$ , Key = 11

compare 11 with 64, 25, 22, 12, Shift 64, 25, 22, 12:

Insert 11: [11, 12, 22, 25, 64]

∴ Sorted array: [11, 12, 22, 25, 64]

Q.3) What is time complexity of insertion and merge Sort.

Sol<sup>n</sup>:- Insertion Sort:

Time complexity:

Best:  $O(n)$ , already Sorted.

Average:  $O(n^2)$

Worst:  $O(n^2)$

Space complexity:  $O(1)$ , in place Sorting.

Merge Sort:

Time complexity:

Best:  $O(n \log n)$

Average:  $O(n \log n)$

Worst:  $O(n \log n)$

Space complexity:  $O(n)$ , temporary array.



Q.4) How does Insertion Sort handles duplicate?

Sol<sup>n</sup>:- Insertion sort is a stable sort algorithm which means that equal or duplicate elements are in their original order.

When it compares each elements with elements in the sorted section (to the left) and inserts it into the correct position and if any duplicate elements are there, they will be placed in the order they appear in the array (original order) which tells it is a stable sort algorithm.

⇒ Example [9, 3, 2, 3]

→ In first iteration  $i=1$ , 3 will get inserted to the left making array [3, 9, 2, 3]

→ In second iteration,  $i=2$ , 2 will get inserted, [2, 3, 9, 3]

→ Now for third iteration,  $i=3$ , Key=3, it is a duplicate element and it will be compared to 9 and 3 and will get inserted at index 2 after the already sorted 3, [2, 3, 3, 9].

Q.5) what are advantages and disadvantages of Insertion sort and merge sort.

Sol<sup>n</sup>:- (i) Insertion Sort:

→ Advantages:

- Stable sort algorithm.
- In place sorting, no extra space.

→ Disadvantages:

- Insufficient for large datasets.
- $O(n^2)$  time complexity.
- Repeated comparing and shifting.



(ii) Merge sort :

→ Advantages:

- Efficient for large datasets.
- $O(n \log n)$  time complexity.

→ Disadvantages:

- Become slow for smaller dataset due to recursion.
- Extra memory space required.
- $O(n)$  space complexity.

Q.6) Write Pseudocode for Insertion Sort.

Sol<sup>n</sup>: function insertionsort (array):

$n = \text{length}(\text{array})$

    for  $i$  from 1 to  $n-1$ :

$\text{Key} = \text{array}[i]$

$j = i-1$

        while  $j \geq 0$  and  $\text{array}[j] > \text{Key}$

$\text{array}[j+1] = \text{array}[j]$

$j = j-1$

$\text{array}[j+1] = \text{Key}$

    return array.

Q.7) Write Pseudocode for Merge Sort.

Sol<sup>n</sup>:- function mergesort (array):

    if  $\text{length}(\text{array}) \leq 1$ :

        return array

$\text{mid} = \text{length}(\text{array}) / 2$

$\text{left} = \text{mergesort}(\text{array}[0:\text{mid}])$

$\text{right} = \text{mergesort}(\text{array}[\text{mid}:\text{length}(\text{array})])$

    return merge (left, right).



```
function merge (left, right):
```

```
    Sorted array = []
```

```
    i = 0
```

```
    j = 0
```

```
    while i < length(left) and j < length(right):
```

```
        if left[i] < right[j]:
```

```
            append left[i] to Sortedarray
```

```
            i = i + 1
```

```
        else:
```

```
            append right[j] to Sortedarray
```

```
            j = j + 1
```

```
    while i < length(left):
```

```
        append left[i] to Sortedarray
```

```
        i = i + 1
```

```
    while j < length(right):
```

```
        append right[j] to Sortedarray
```

```
        j = j + 1
```

```
    return Sortedarray.
```

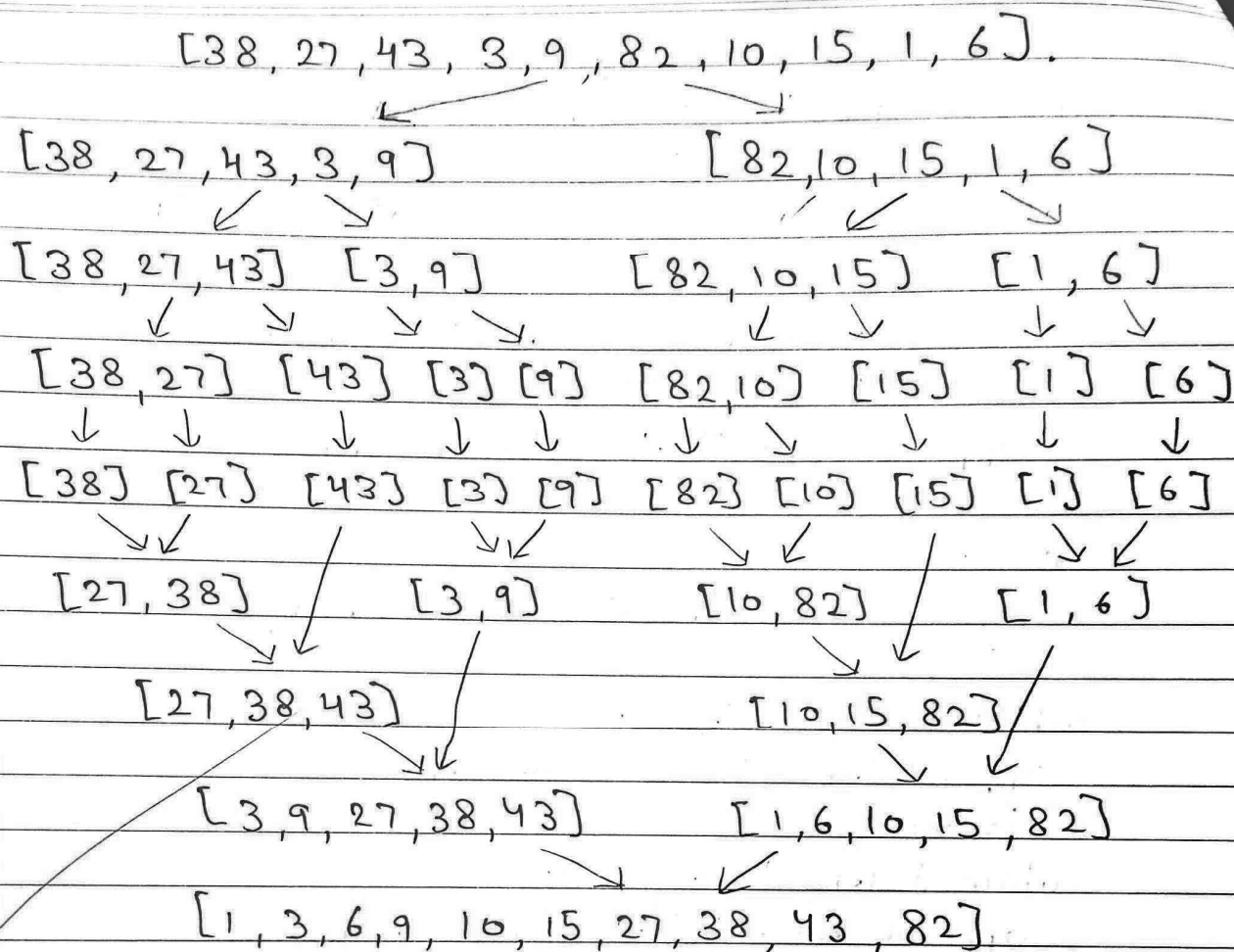
Q.8) Sort the elements [38, 27, 43, 3, 9, 82, 10, 15, 16]

Using Merge Sort.

Sol<sup>n</sup>:- Merge sort using recursion and divide, conquer, merge for the sorting.

First it will divide the array recursively into two halves until it cannot be divided and each subarray is individually sorted using merge sort algorithm and are merged back together in the sorted order.





∴ final sorted array: [1, 3, 6, 9, 10, 15, 27, 38, 43, 82]

\* Conclusion:

Thus, we have implemented Insertion Sort,  
Merge Sort in C++ using an array.